

1 - Analysing Simulink

Donnerstag, 24. Juli 2025 15:34

Datum	24.07.2025
Kategorie	Tool Analysis
Quelle	MATLAB & Simulink
Arbeitspaket	Specification - Analysis Existing Tools

1 Concepts: System, Model, and Components

- **System:** Interdependent parts with state evolution over time.
- **Model:** Mathematical abstraction (usually DAEs or difference equations).
- **Component:** Modular part with input/output interfaces.
- **DAE Systems:** Often consist of differential and algebraic parts (algebraic loops reflect algebraic constraints).

2 Model Structure in Simulink

Element	Description
Block Diagrams	• Visual interface. • Made up of blocks, signals, events, and data. • Hierarchical (includes Subsystems and Model blocks)
Blocks	• Represent behavior via block methods (derivative, update, output). • Virtual (for organization) or Non-Virtual (for execution control). • Atomic subsystems: isolated, compiled as single units
Signal and Communication Lines	• Signal lines: continuous/discrete time-varying values. • Event lines: for triggering blocks/subsystems (e.g. via Stateflow) • Message lines: for asynchronous, event-based communication
Parameters and Data	• Reside in model workspace, base workspace, or data dictionaries • Can be tuned during simulation (tunable parameters)
States and Time Behavior	• Continuous and discrete states • Hybrid systems: require solvers that coordinate across different sample times • Sample time governs when blocks execute
Algebraic Concepts	• Direct feedthrough: Output depends directly on current input. • Algebraic loop: Circular dependency among direct-feedthrough blocks. • Zero-crossing detection: Helps precisely capture discontinuities.

3 Simulink Simulation Pipeline

3.1 Model Compilation Phase

1. Evaluates block parameter expressions
2. Determines signal attributes (type, dimension, etc.)
3. Propagates signal attributes to connected blocks
4. Performs optimizations (e.g., block reduction)
5. Flattens virtual hierarchies
6. Sorts execution order (task-based)
7. Resolves sample times

- Outputs:
 - execution order
 - memory structure
 - sample times
 - metho execution lists

3.2 Link Phase

- Allocates Memory for runtime simulation
- Builds execution lists for block methods (Outputs, Update, Derivatives)
- Initializes SimBlocks structures (buffers, state pointers)

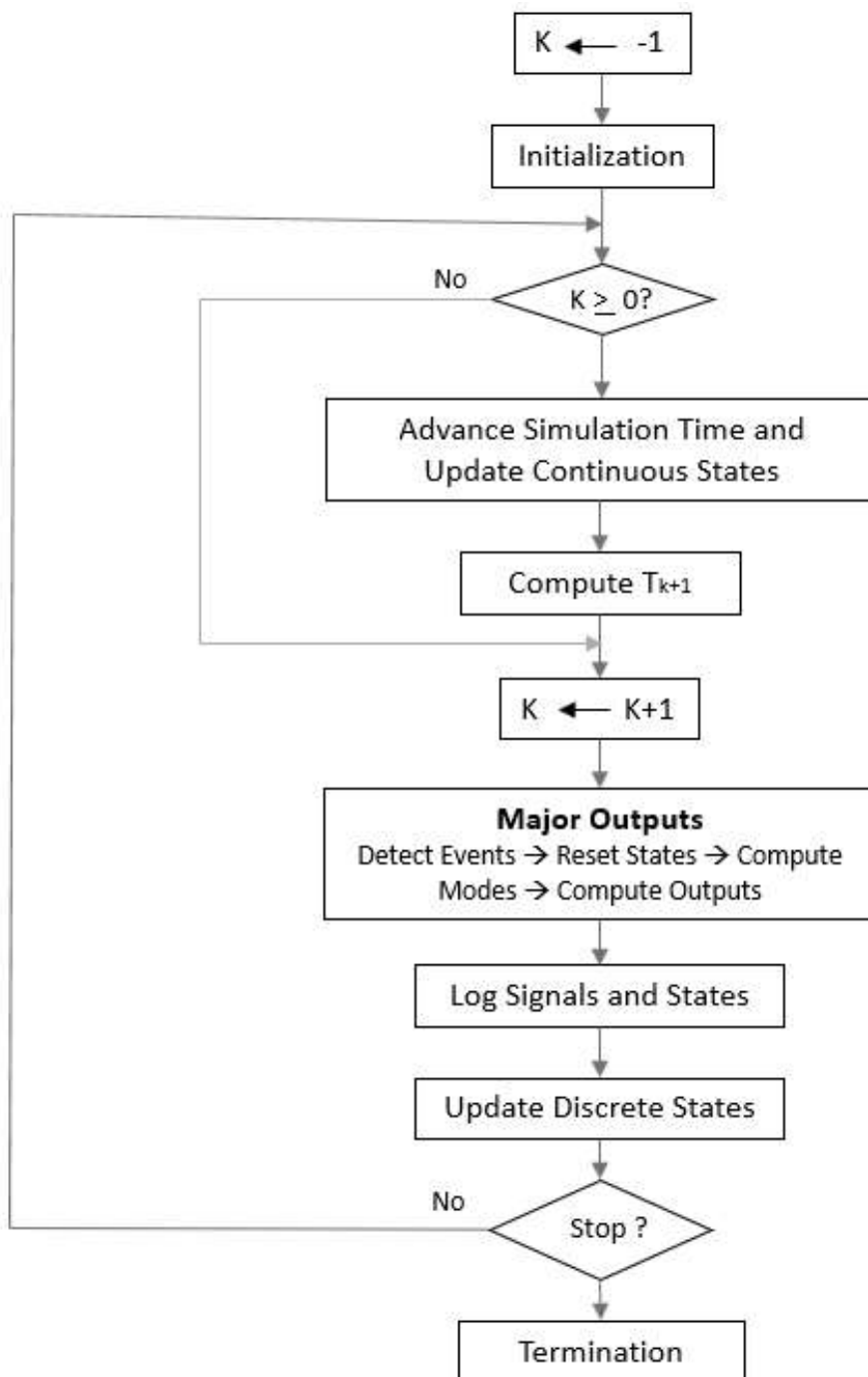
3.3 Simulation Loop Phase

- Engine successively **computes the states and outputs of the system at intervals** from the simulation start time to the finish time
- Step size depends on the type of solver, the system's fundamental sample time, and whether the system's continuous states have discontinuities (zero-crossing detection)
- Iteration phase is repeated once per time step from the simulation start time to the simulation stop time
- Each step: new values for the system's inputs, states, and outputs are computed
 - model is updated to reflect the computed values

Loop Iteration

No.	Step	Description
1	Computes model outputs	• engine initiates this step by invoking the simulink outputs method • model -> system -> output methods of blocks • in the order specified by the outputs method execution lists generated in the link phase of the simulation • system outputs method passes the following arguments to each block output methods: • a pointer to the block's data structure • a pointer to its SimBlock structure • SimBlock data structure points to information that the outputs method needs to compute the block's outputs (location of its input and output buffers)
2	Computes model states	• model's states are computed by invoking a solver • Discrete states: discrete solver is onvoked • step size is computed to meet the model's sample times • then invokes the Update method • model -> system -> block that the system contains ◦ order specified by the Update method lists generated in the Link phase • Continuous states: engine invokes the continuous solver specified by the user • depending on the solver, ◦ the solver either in turn calls the derivatives method of the model once or ◦ eneters a subcycle of minor time steps where the solver repeatedly calls the model's Outputs methods and derivatives methods to compute the model's outputs and derivatives at successive intervals within the major time step • Model outputs methods and derivatives methods ◦ -> invoke their corresponding system methods ◦ -> invoke the blocks outputs and derivatives
3	Check for discontinuities	• Zero-crossing detection is used to detect discontinuities incontinuous states

4	Compute new time for next time step	• Steps 1 to 5 are repeated until the simulation stop time is reached
---	-------------------------------------	---



Fundamental Course

Freitag, 20. Juni 2025 08:45

Datum	19.06.2025 bis 23.06.2025
Kategorie	Tool Analysis / Framework Inspiration
Quelle	Simulink Fundamental Course, Simulink Documentation
Arbeitspaket	AP4: Analyse bestehender Tools

Summary:

- Core Modeling Concepts
 - Continuous and discrete systems modeled via ODEs, difference equations, transfer functions, state-space models
 - Block-based modeling: visual hierachies and data flow structures
 - Look-up tables: interpolated, multidimensional modeling of functions
 - Virtual and atomic subsystems help with modularization and execution control
- Simulation Management
 - Solvers handle the numerical integration (fixed and variable step)
 - Performance tuning via solver profiler and zero-crossing detection
 - Conditional execution for event-driven modelling: enabled and triggered
- Data Handling & Integration:
 - Exporting and importing of data: .mat, workspace, spreadsheets
 - Signal organization via Bus signals
 - Scope and data inspector for output analysis
 - Model referencing: supports scalability, modularity
- Testing & Modularity:
 - Model referencing enables reusable component design with strong encapsulation
 - Support for unit testing, integration testing, and simulation comparison
 - Workspaces isolate and mangle parameters for modularity and reuse
- User Interface & Visualization:
 - Masking abstracts the complexity and exposes parameters in a dialog
 - Live analysis via scope, port labels, data inspectors
 - Dependency analyzer maps model references and hierachies

Relevant Concepts:

Visual hierachy / subsystems	• Submodel encapsulation • Modular simulation blocks (Python classes for subsystems like MotorSubsystem) • Subsystem templates with input configuration-file
Atomic Subsystems	• Execution controlled modules • For FMI components
Bus Signals	• Typed and grouped signal interface
Workspaces	• Base, model, mask • Scoped configuration for project, model, and component level
Conditional subsystems	• Event-driven execution • Trigger-based simulation paths
Model referencing	• Modular and reusable submodels
Masking	• Parameter dialog • Differntiation of standard- vs. Expert.user interface

Scopes / Data Inspectors	• Live plotting and result comparison tool • Integration of real-time insights (matplotlib, Plotly)
Architecture	• Layered architecture • Clean separation between model description, simulation engine, and user interaction
Tools	• Debugging complex models with dependency analysis tool • Simulation profiling: identifying performance bottlenecks • SSA: approximation models

Simulink Fundamental Course

Lookup table:

- Representations of physical systems
- Performs n-dimensional interpolated table lookup including index searches
- Table is a sampled representation of a function in N variables
- Breakpoint sets relate the input values to positions in the table
- First dimension corresponds to the top (or left) input port
- Extrapolates values as well
- Allows to choose the Number of table dimensions and then enters multiple sets of breakpoint data
- 2D lookup table would define a surface for lookup values

Callbacks and Initialization Functions

- Enables the definition of functions and parameters independent of the matlab workspace

Data Input/Output

- .mat files
- Workspace variables
- Ports to create hierarchy with other models
- Spreadsheet data

Import / Output

- Full specification of data type and dimensions
- Reading input data for a model [t, u1, u2]
- Serve as root level output and interface with other levels of model hierarchy

View and Analysis of Simulation Results

- Scope Block
 - Visualization and measurement capabilities
 - Similar to benchtop oscilloscope
- Simulation Data Inspector
 - compare results across simulation runs
 - Import data from a previous session
- XY-Graph
 - XY Plot of two signals
 - Signals are automatically logged and can be also viewed using the simulation data inspector
- Logging Data to Data Inspector
- Port Value Label - live plotting of intermediate signals

Modeling Dynamic Systems

Continuous Dynamic Systems

Modeling Differential Equations

- Continuous internal states that evolve over time
- States' time derivatives are used to describe the rate of change

Example:

$$C \frac{d^2 V}{dt^2} + \frac{1}{R} \frac{dV}{dt} + \frac{1}{L} V = 0$$

1. Arrange the equation so the highest order derivatives are isolated on the left-hand side of the equation, and the remaining terms are on the right side

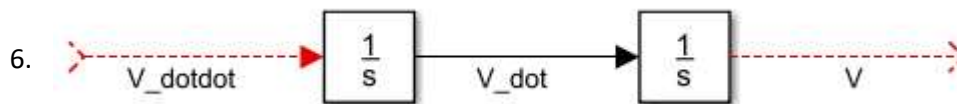
2.
$$\frac{d^2 V}{dt^2} = \frac{1}{C} \left(-\frac{1}{R} \frac{dV}{dt} - \frac{1}{L} V \right)$$

3. Determine the number of integrator blocks requires

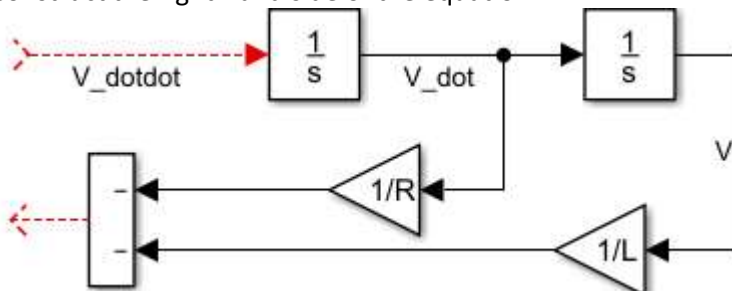
$$\frac{d^2 V}{dt^2} \rightarrow \frac{dV}{dt}$$

4.
$$\frac{dV}{dt} \rightarrow V$$

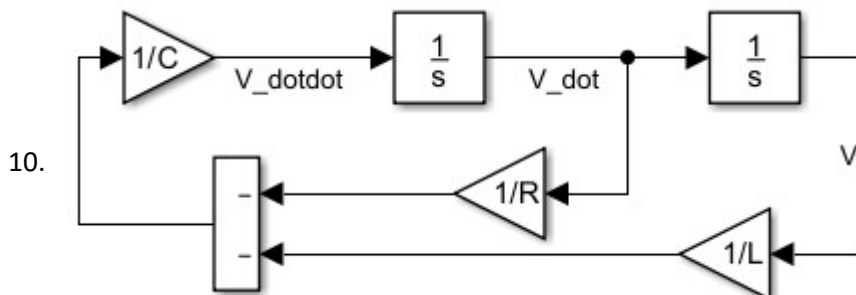
5. Add the integrator blocks and label their signals



7. Construct the right-hand side of the equation:



9. Connect the remaining signals, as necessary, to finish the equality



11. Set the initial conditions

Continuous Transfer Functions

- Compact representation of the differential equations for linear time-invariant (LTI) systems
- State-Space-Representations models a system using a set of first-order differential (or difference) equations that describe the system's dynamics in terms of state variables

General Form (continuous-time):

$$\begin{aligned}\dot{x}(t) &= Ax(t) + Bu(t) \\ y(t) &= Cx(t) + Du(t)\end{aligned}$$

- $x(t)$: State vector (describes the system's internal state)
- $u(t)$: Input vector
- $y(t)$: Output vector
- A, B, C, D : System matrices

Example:

For the same mass-spring-damper system:

Define $x_1 = x$, $x_2 = \dot{x}$. Then:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1/m \end{bmatrix} u(t)$$

Output equation:

$$y(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

- Describes both internal dynamics and input-output behavior
- Handles MIMO (Multiple Input Multiple Output) systems naturally
- Can represent time-varying and non-linear systems (in extended forms)
- Easily accommodates initial conditions

Simulink Solvers

	Continuous solver	Discrete solver
Variable - step	For simulating and developing most applications	For simulating and developing controllers, filters...
Fixed - step	For hardware-in-the-loop simulations, code generation...	For deploying controllers, filters...

Discrete Dynamic Systems:

- System's dynamics are expressed in the form of difference equations
- Separate equations are used to compute the system's outputs and update the system's states
 - Output difference equation: describes the system's output in terms of its discrete state $x(k)$ and input $u(k)$
 - A state update difference equation describes how a discrete state $x(k)$ changes based on previous state values $x(k-1)$ and input $u(k)$
 - Index variable k represents the number of discrete time steps taken and is related to the simulation time

Example:

$$x[k] - x[k-1] = u[k] - u[k-1]$$

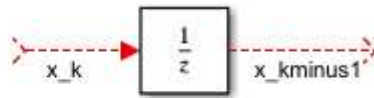
$$u[k] = \sin[k]$$

1. Arrange the equations so the most recent state is on the left hand side of the equation, and the remaining terms are on the right:
$$x[k] = x[k-1] + u[k] - u[k-1]$$
$$u[k] = \sin[k]$$
2. Determine the number of unit delay blocks required.

3.
$$\begin{cases} x[k] \rightarrow x[k-1] \\ u[k] \rightarrow u[k-1] \end{cases}$$

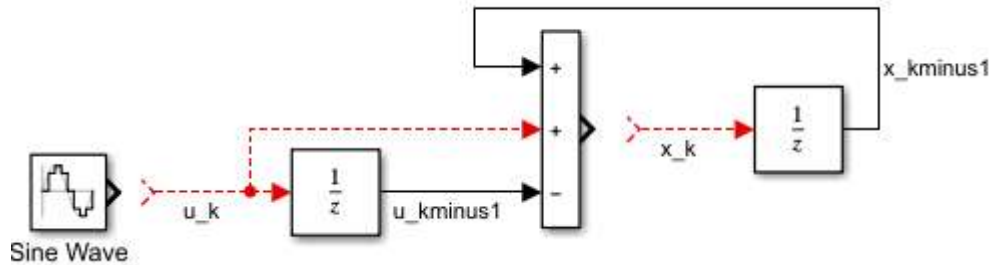
4. Add the unit delay blocks and label their signs

5.



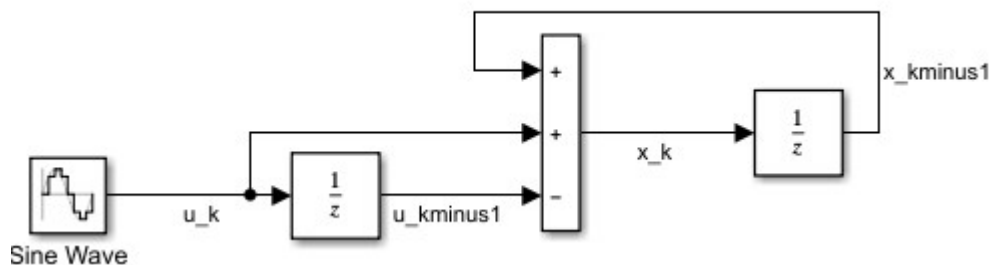
6. Construct the right side of each equation

7.



8. Connect the remaining signals, as necessary, to finish the equality

9.



10. Finally, set all sample times and initial conditions

Subsystems

- Creating a visual hierarchy
- Subsystems to improve model's scalability and readability
 - Keeping functionally related blocks together
 - Establishing hierarchy
 - Reducing the number of blocks displayed at a time
- Inports and Outports allows information to pass between the subsystem and its parents
- Virtual subsystems creates a visual hierarchy without affecting model execution
- Subsystem boundaries act as visual aids
- Model with **virtual subsystems**: Execution order is the same as if it were a flat model
- **Atomic subsystems**: subsystem is **treated as a single unit in simulation** and can appear as individual functions when generating code from a simulink model

Bus Signals

- Keep signal lines organized by grouping related signals
- Vector Concatenate is useful for simple vector math operations
 - Requires the grouped signals to have the same data type and dimensions
- Signal buses allow to:
 - Name signals
 - Combine signals of different data types
 - Ensure that signals are contiguous in memory
- Bus creator to merge signals within a bus
- Bus selector to separate signals from a bus

Masking

- Reuse of subsystem: parameters are shared
- Parameters must not be defined in the base workspace

- Parameters of the subsystem are exposed as editable mask parameters
- Informative mask icon: displaying parameters or adding features
- Help documentation for subsystem mask
- Allows to create a parameter dialog box for the block, enabling users of the block to easily change relevant parameters just by double-clicking the box
- Mask workspace is distinct from the model or base workspace: enables reusability through parameterization
- Masks can also shield users from complexity they do not need to see

Conditional Block Execution:

- Execution of blocks should depend on the execution of other components
- Example: require a block to update in response to a value elsewhere in the model
- Subsystems that execute based on a condition signal input -> control signal attached to a control input
- **Conditionally executed subsystems must be atomic**
 - Nonvirtual hierarchy and boundaries of atomic systems are necessary to enforce the proper execution
- Main types of conditionally executed:
 - **Enabled**
 - **Triggered**
 - Refers to the way the control signal affects the execution

Enabled Subsystems

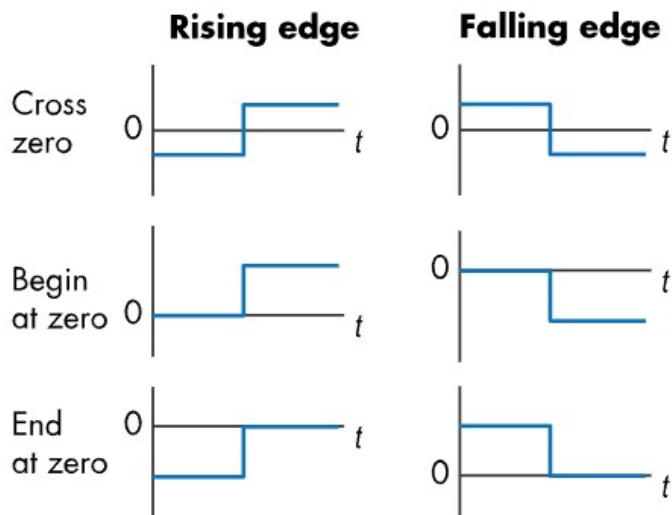
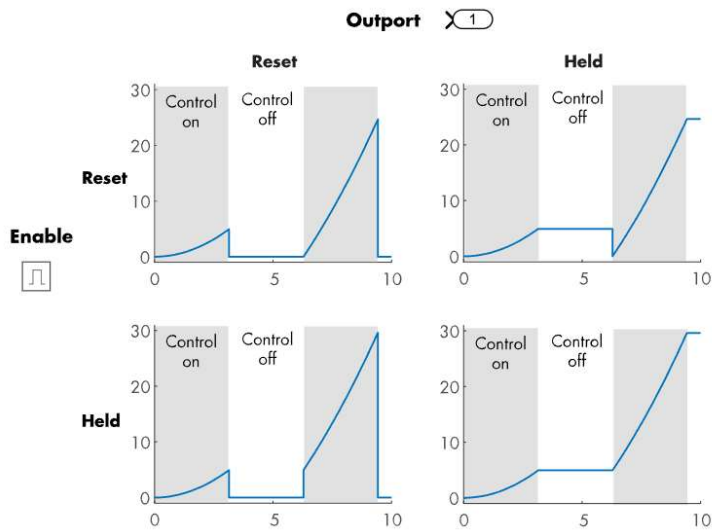
- Depending on the control signal subsystem can be enabled or disabled
- Enabled: subsystem executes based on its sample time -> producing its output and updating its state variables
- Disabled: subsystem does not execute -> behavior of the subsystem outputs and states is dependent upon options set in the enable or Output blocks
- Is enabled whenever the control signal input is greater than 0
- Can contain discrete and continuous blocks

Triggered Subsystems

- Only execute when a trigger event occurs
- Trigger event occurs when the control signal crosses zero in some way
- Outputs and states of a triggered system are always held to their previous values between trigger events
- Can only contain discrete blocks
- Sample time must be set to inherited

Enabled and Triggered Subsystems

- Subsystem can have enable and trigger control signal
- Can be used to validate the execution of an event driven system with a condition
- Trigger while enabled -> behaves like a triggered subsystem
- Trigger while disabled -> behaves like a disabled subsystem



Integrating Multiple Models

- Interface requirements
- Testing

Model Referencing Workflow

- Allows to include a simulink model within another model as a block
 - Useful in large system hierarchies that require collaboration within a team
 - System design is typically driven by specifications such as system requirements
1. Use system requirements to create
 - a. model hierarchy
 - b. inputs/outputs
 - c. Data
 - d. configuration parameters, etc.
 2. Build the somponlink blocks
 3. Unit Testing: Verify that the individual design works correctly and satisfies the requirements
 4. Integrate the component into ist parent model or models and make sure it behaves as expected
- Managing of model data and configuration parameters has to be considered

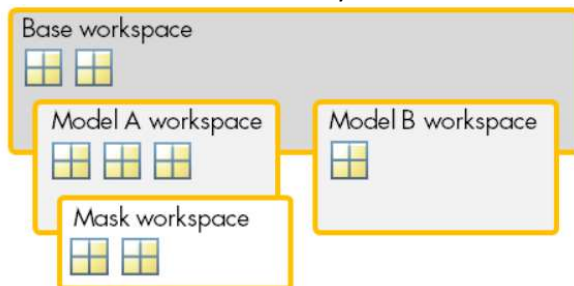
Model Referencing Advantages

- Modular development

- Components can be designed, modeled, and tested independently
- Concurrent workflows
 - teams can work simultaneously on components in separate files
- Model protection
 - Data types at component boundaries can be strictly enforced
- Scalability
 - Model can be referenced multiple times and in multiple models
- Incremental loading
 - Referenced models are not loaded until they are needed
 - Reducing amount of memory needed by the parent model
- Performance
 - Code generation improves simulation performance of referenced models using accelerator modes

Workspaces

- Management of parameter and model configurations become more critical for larger models
- Workspaces to store model data
- **Base** Workspace
 - Parameters are global and can be used or modified by any model
- **Model** Workspace
 - Parameters are local to the model and cannot be used directly by other models
- **Mask** workspace
 - Local to the subsystem that is masked



Model Explorer

- Is used to manage model data
- Enables to modify workspaces, model elements, and model configurations
- Useful when working with multiple models
- Allows to access every model from a single interface

Dependency Analyzer

- Opens and analyzes the model
- Displays a dependency graph of the models that the parent model references
- Nodes in the graph represents models
- Directed lines indicate model dependencies
- Models can be opened via the graph

Optimizing Simulation Performance

- Solver numerically approximates the solutions to differential equations represented by the simulink model
- Euler method = ode1 fixed step solver
- Ode3 and ode4 with minor steps within a single simulation step
- Performance Solver Profiler
 - Suggestions on maximum and minimum step size
- Simulation of Discontinuities
 - Logical conditions, saturation block

- Dynamics of the physical system (switch or relay in circuit)
- Zero-crossing can be detected and re-simulation with finer step-size

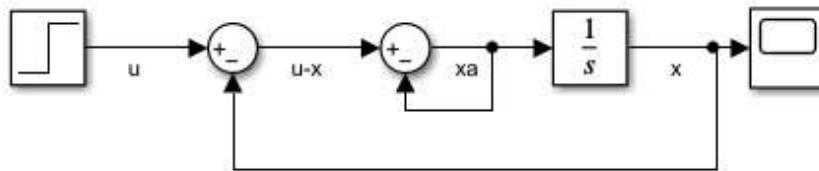
Simulink Concepts

Donnerstag, 24. Juli 2025 16:54

Date	24.07.2025
Category	Tool Analysis
Source	MATLAB & Simulink

1 System and Model

- System is a group of interdependent physical and functional parts with measurable characteristics that change over time.
- System component is a part of a system that interacts with other parts of the system. The interactions between the components define the structure and behavior of the system.
- A model is a mathematical description of a system derived from physical laws or experimental data. The description typically uses a set of variables with a set of differential and difference equations that define the relationship between these variables
- A model component is a part of a model that interacts with other parts through an interface of inputs and outputs. Simulink implements model components using Subsystem and Model blocks. A Model block references another simulink model saved in a separate file
- Differential algebraic equations (DAEs) contain additional constraints that involve the independent state variable and the state vector in addition to differential equations.
 - algebraic = equations that do not involve derivatives
 - algebraic loops represent algebraic constraints
 - models with algebraic loops define a system of differential algebraic equations
 - Inner loop: algebraic constrain
 - Outer loop: differential equation



The model implements this system of DAEs.

$$\dot{x} = x_a$$

$$0 = u - x - 2x_a$$

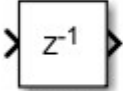
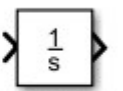
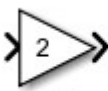
2 Simulink Models

2.1 Block Diagram

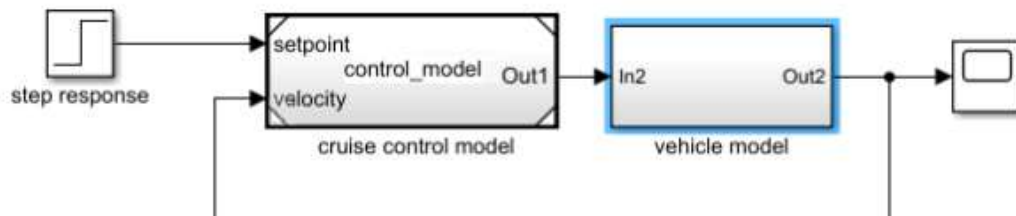
- Visual representation of a model in the editor (allows to add blocks from block libraries)
- Blocks are connected to each other with signal and event lines
- Block within a block diagram itself defines a model in itself
- Block methods define the time-based relationships between input signal, output signals and block state variables
- Parameters define the coefficients within a block

2.2 Blocks

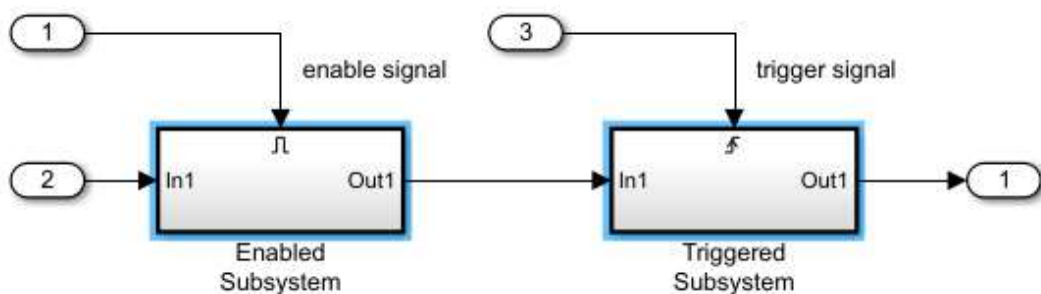
- Basic modeling construct (input signals, output signals, and states)
- **Parameters** describe the block behavior
- Each block represents a set of equations for the simulink engine (block methods)
- **Block methods** are evaluated during the simulation of a block diagram within a simulation loop

Blocks	 Delay  Integrator  Gain  Constant
Block Methods	$\begin{cases} x[n+1] = u[n] \\ y[n] = x[n] \end{cases} \quad \begin{cases} \frac{dx}{dt} = u(t) \\ y(t) = x(t) \end{cases} \quad y(t) = 2 * x(t) \quad y(t) = 5$

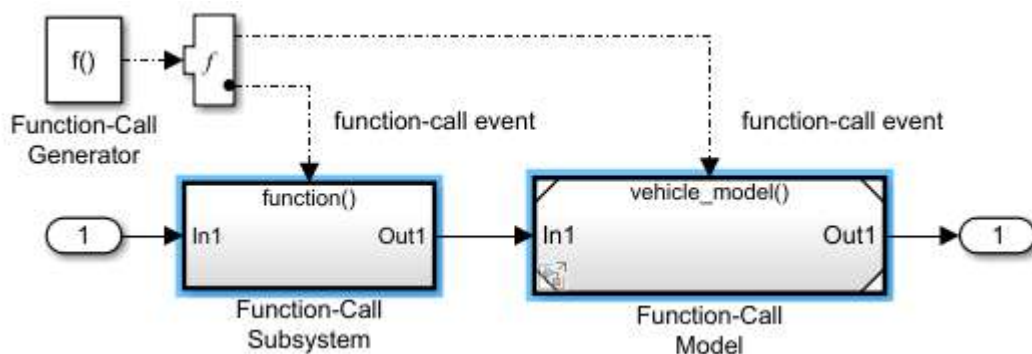
- **Virtual blocks** organize and provide hierarchy in a model
 - Flattening: During model simulation the blocks are expanded in place before execution(similar to macros in C or C++)
 - Can be treated as atomic unit
 - Inports and outports are used to exchange data with a subsystem block
 - Bus blocks combine signals into a virtual bus and manage the routing of signals around a complex block diagram
- **Nonvirtual blocks** provide execution control and graphical hierarchy
 - **Atomic** subsystem and models are atomic executed (grouping functional aspects of the model)



- **Enabled/Triggered:** Atomic Subsystems that are controlled by external signal



- **Function-Call:** Atomic Subsystems whose execution is controlled by an event from an function-call initiator

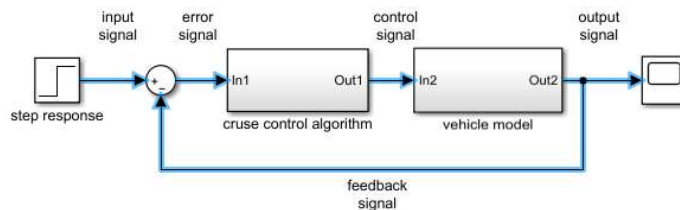


- **Action Subsystem:** atomic block whose execution is controlled by an event from an action initiator (switch case block)
- **Iterator Subsystem:** runs multiple iterations during each model time step (contains while or for loop)
- **Block Mask:**
 - custom block parameter interface that displays only selected block parameters
 - Provides interface for setting parameters on blocks inside the subsystem block without having to navigate the model hierarchy

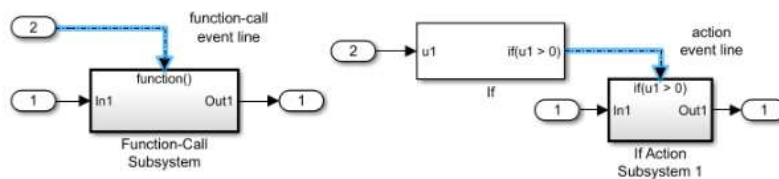
- **Custom Blocks:**
 - new blocks that extend the built-in functionality
 - libraries of new blocks can be created

2.3 Lines

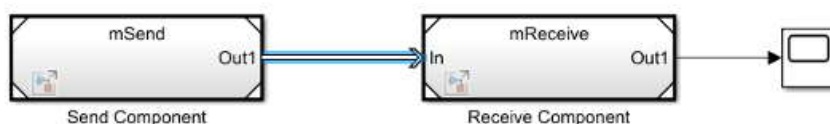
- Connecting output ports of blocks with the input ports of blocks
- **Signal lines**
 - transfer data during simulation from one block to another
 - Signals are time-varying quantities that have values at all points in time (continuous) or at specified time points (discrete)
 - Source of a signal: block that writes to the signal during evaluation of its block output method
 - Destination: block that reads the signal during evaluation of its block input method
 - Save data from a signal by logging the signals



- **Event Lines**
 - send an event to a subsystem or model
 - block that receives the event executes the blocks within it one or more times during a time step
 - Function-call initiator include Stateflow charts, function-call generators, S-functions, hit crossing (include if ans switch case blocks)



- **Message Lines**
 - indicates a path along a message can advance within a distributed system
 - used to model asynchronous event-based communication between components for the purpose of system integration



2.4 Data

- Includes the parameters and input signal values that are used to produce the output values resulting from simulating the behavior of the model
 - Model parameters: specify block and signal parameters
 - Model configuration parameters: settings that control model behavior
 - Simulation data: input data to drive a simulation and output data generated
 - Data objects: allow to specify characteristics of signals, states, and block parameters
- Location of data
 - block parameters
 - base workspace (global)
 - model workspace (model namespace)
 - data dictionary to define and store global data
 - imported and exported data

2.5 Parameters

- Model characteristics that affect the simulation and code generation result

- Model configuration parameters
 - specify the model behavior during compilation, simulation and code generation (solver)
- Block Parameters: define model dynamics and mathematics
- Block Parameter Variables:
 - variable value is used as the parameter values (reference)
 - from model workspace (use simulink parameters for data type, dimension and unit)
 - from data dictionary
 - base workspace (same set of parameters for more than one model)
- Model argument and instance parameter
 - referencing the same model from multiple model blocks -> create instances to use the same of different value for each instance
- Tunable block parameters:
 - change the value during a simulation
 - determine interactively the most suitable value for a parameter
 - change take effect at the start of the next time step (gain parameter)

2.6 Properties

- Model characteristics that generally do not affect the simulation result
- Model properties (general, design data, callbacks when a specific event occurs)
- Block properties (annotation, callbacks, priority, tags)
- Port properties
- Model and block properties are set with property inspector

2.7 State Variables

- State of a model is described by the values of its state variables
- State variables are the set of variables whose values at time zero, along with the model inputs and the model equations, can determine the behavior of a model during a simulation
- If the current output value is a function of the previous output value, the block defines a state variable that needs to be saved between time steps (saving value of the state variable at a current time step for use in computing the output at a subsequent time step)
- Working with states
 - specify initial values of model states, record state values
- **Continuous states:** defined for all values of time (Integrator, state-space, transfer-fcn)
- **Discrete states:** defined only at specific times
 - approximation of a continuous state (periodic or aperiodic update)
 - Discrete-time integrator, discrete state space, discrete transfer fcn
 - delay
- **Modeling Hybrid Systems**
 - has continuous and discrete states
 - solving entails choosing a step size that satisfies both the precision constraints on the continuous state and the sample time interval constraint on the discrete state
 - passing the next sample time interval for the discrete solver as an additional constraint on the continuous solver
 - continuous solver chooses step size that advances the simulation up to but not beyond the time of the next discrete time sample
- **Sample Time:**
 - time interval that specifies the rate for executing block methods to produce outputs and update internal states of block
 - Continuous: blocks run at variable times based on solver settings
 - Discrete: blocks run at specific times that are explicitly specified
 - Block Sample Time: time which defines when the block executes
 - Discrete sample time: time is a vector (discrete block only allows discrete sample time)
 - continuous sample time: nominal sample time (state changes continuously)
 - inherited: simulink determines the actual value based upon the context of the inherited block within the model

- Allows to see the different sample times within a model (View sample time information)

2.8 Units

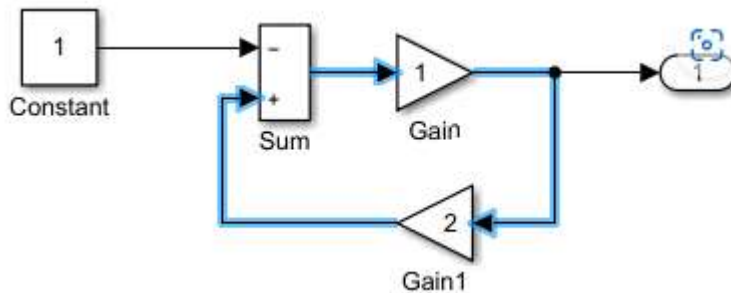
- Units are used to measure the total amount of a quantity
- Units are specified as inportt/output block parameter at the boundaries of a simulink model component

2.9 Direct Feedthrough

- Output port signal of a block is computed from the values of its input port signals
- Signal value is a function of the input values
- Examples: Gain, Product, Sum, Math Function Blocks

2.10 Algebraic Loop

- Signal loop between blocks with direct feedthrough
- Occurs when an input port of a block with a direct feedthrough is driven directly by the output port of the same block or indirectly through other blocks with direct feedthrough



2.11 Artificial Algebraic Loops

- Occurs when an atomic subsystem or model causes simulink to detect an algebraic loop, even though the contents of the subsystem do not contain a direct feedthrough from the input to the output

2.12 Zero-Crossing Detection

- Simulink uses zero-crossing detection to accurately locate a discontinuity without resorting to excessively small time steps during a simulation
- Technique improves simulation run times

3 Simulink Simulation

3.1 Compilation

- Process where the block diagram is translated to an internal representation that interacts with the Simulink Engine
- There are no model level equations that are solved numerically as a whole
 - model-level equations correspond to the individual block equations that are solved numerically in a specific order
- Block Methods
 - functionality of a block is defined by multiple equations (represented as block methods)
 - Block methods are evaluated (executed) during the execution of the block diagram
 - Evaluation of these block is performed within a simulation loop (each cycle through the loop represent the evaluation of the block diagram at a given point in time)
- Common Block Methods:
 - Derivative: computes the derivative of the block's continuous states at the current time step (with given input and state at previous time step)
 - Update: computes the values of the discrete states at the current time step (with given input and discrete state at previous time step)
 - Output: computes the outputs of a block given its input at the current time step and its states at the previous time step
- Model Method
 - compute the model properties and its outputs

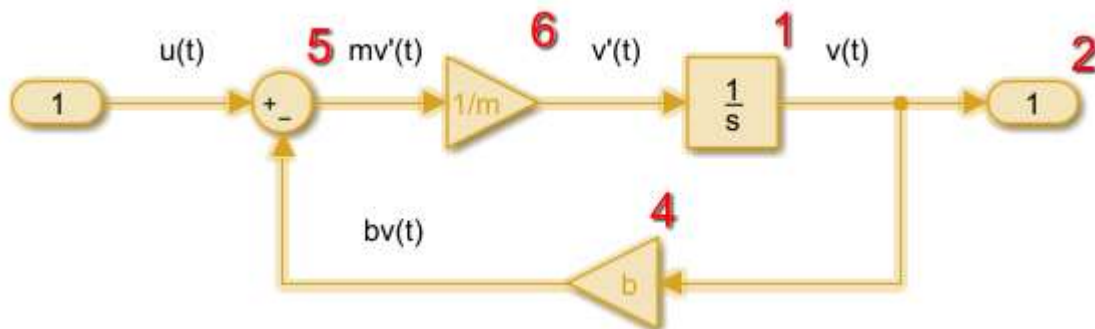
- to determine a model's properties and its outputs
- model method invokes the block methods of the same type
- For example, the model Outputs method invokes the Outputs methods of the blocks that it contains in the order specified by the model to compute its outputs
- The model Derivatives method similarly invokes the Derivatives methods of the blocks that it contains to determine the derivatives of its states.

3.2 Callback

- Expressions that execute in response to a specific modeling action
- You provide the code for callback parameters, Simulink executes the callback code when the associated modeling action occurs
- Model Callback: PreloadFcn
 - executes before a model load (provide code that loads the variable values a model uses into the Matlab Workspace)
- Block Callback:
 - OpenFcn: execute when you open a Subsystem block
 - LoadFcn: execute after a diagram is loaded
- Port Callback:
 - ConnectionCallback: execute code every time the connectivity of a port changes

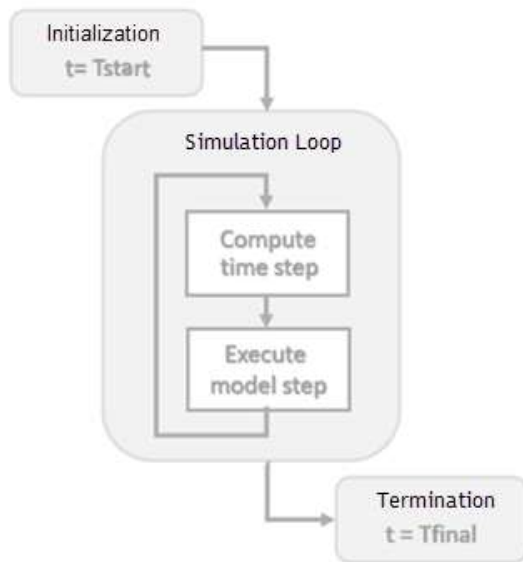
3.3 Execution Order

- Execution order is the sequence in which block outputs are called after evaluating direct feedthrough of each input port
- Integrator block must run first, then the loop of blocks connected to the integrator block
- Missing execution numbers are usually due to so-called hidden buffer#



3.4 Simulation

- Simulation is the process after model compilation where block method outputs and states are computed at successive time steps over a specified time range using a numerical solver.
- During each simulation loop, Simulink calculates a Δt to determine the time step $t(k+1) = t(k) + \Delta t$.
 - The size of Δt is based on an estimated error between the simulated solution and the actual solution.
 - At the end of a simulation, data results are given as vectors $[t, X, Y]$ for time, state and output at each time step.



3.4.1 Model Compilation

- Running the simulation causes the simulink engine to invoke the model compiler
- Model compiler converts the model to an executable form -> compilation

Tasks of the compiler:

1. Evaluates the model's block parameter expressions to determine their values
 2. Determines signal attributes (name, data type, numeric type, dimensionality), not explicitly specified by the model and checks that each block can accept the signals connected to its inputs
 3. Propagates the attributes of a source signal to the inputs of a block that it drives in order to compute previously unspecified attributes in the block
 4. Performs block reduction optimization
 5. Flattens the model hierarchy by replacing virtual subsystems with the blocks that they contain
 6. Determines the block-execution order by task-based sorting
 7. Determines the sample times of all blocks in the model whose sample time are not explicitly specified
- Simulink starts model compilation as part of model simulation (compilation leads directly into a linking phase)

3.4.2 Link Phase

- Simulink engine allocates memory needed for working areas (signals, states, run-time parameters) for execution of the block diagram
- Allocates and initializes memory for data structures that store run-time information for each block
- Built-in Block: SimBlock -> stores a pointer to a block's input and output buffers and a state and a work vectors
- Method Execution Lists
 - list the most efficient order in which to invoke a model's block methods to compute its outputs
 - block execution order lists generated during the model compilation phase are used to construct the method execution lists
- Block Priorities
 - update priorities can be assigned to blocks
 - Output methods of higher priority blocks are executed before those of lower priority blocks
 - priorities are honored only if they are consistent with its block execution order

3.4.3 Simulation Loop Phase

- In this phase the engine successively computes the states and outputs of the system at intervals from the simulation start time to the finish time
- States and outputs are computed at time steps

- Step size depends on the type of solver, the system's fundamental sample time, and whether the system's continuous states have discontinuities (zero-crossing detection)
- Loop initialization phase at the start of the simulation
- Iteration phase is repeated once per time step from the simulation start time to the simulation stop time
- Start of the simulation: model specifies the initial states and outputs of the system to be simulated
- Each step: new values for the system's inputs, states, and outputs are computed
 - model is updated to reflect the computed values
- End of simulation: model reflects the final values of the system's input, states, and outputs
- Simulink provides data display and logging blocks

Loop Iteration:

1. Computes the model outputs

- engine initiates this step by invoking the simulink outputs method
- model outputs method -> model system outputs method -> output methods of blocks that the model contain (in the order specified) by the outputs method execution lists generated in the link phase of the simulation
- system outputs method passes the following arguments to each block output methods:
 - a pointer to the block's data structure
 - a pointer to its SimBlock structure
 - SimBlock data structure points to information that the outputs method needs to compute the block's outputs (location of its input and output buffers)

2. Computes the model's states

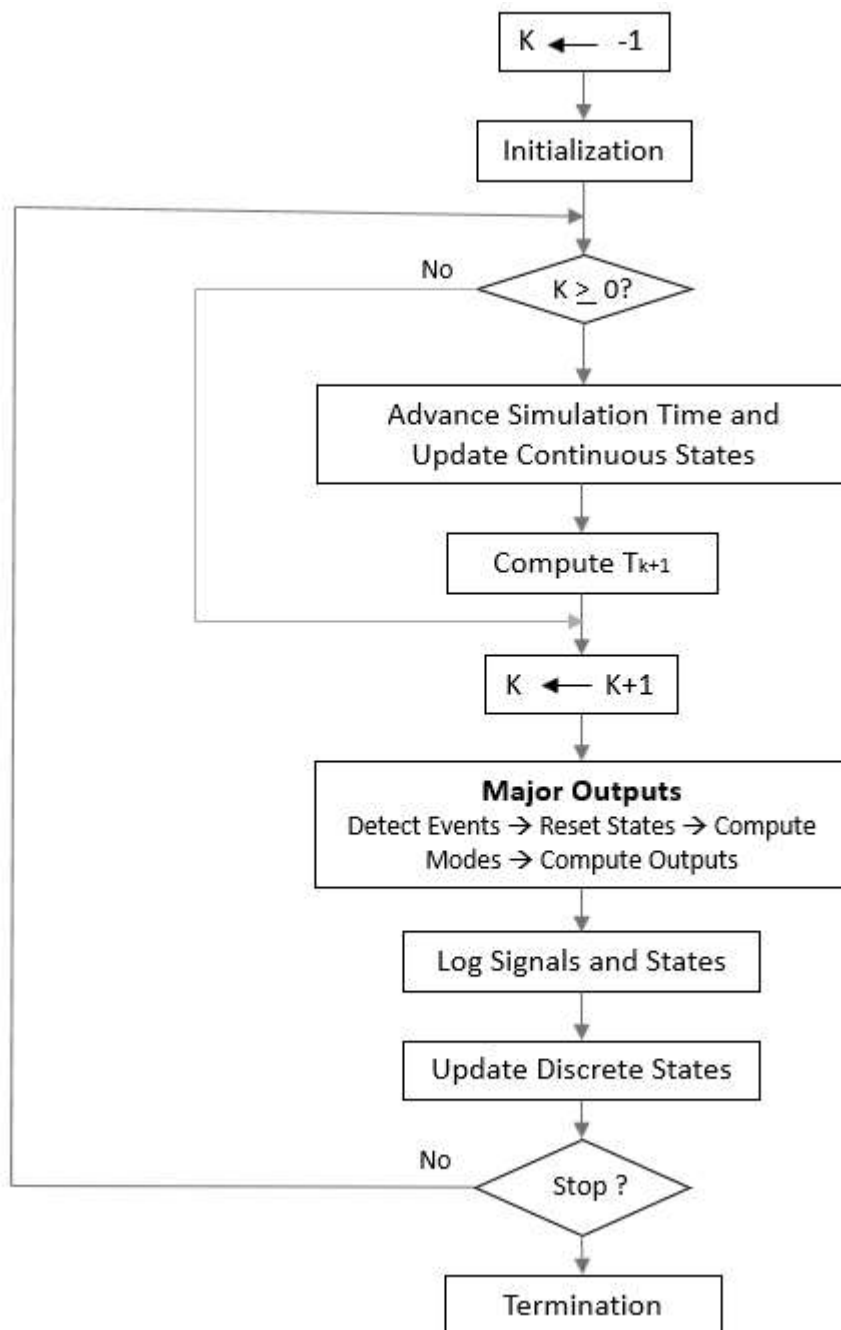
- model's states are computed by invoking a solver (selected solver depends on type and mix of state variables)
- Discrete states: discrete solver is invoked
 - step size is computed to meet the model's sample times
 - then invokes the Update method of the model -> invokes update method of its system -> invokes the update methods of each block that the system contains (order specified by the Update method lists generated in the Link phase)
- Continuous states: engine invokes the continuous solver specified by the user
 - depending on the solver,
 - the solver either in turn calls the derivatives method of the model once or
 - enters a subcycle of minor time steps where the solver repeatedly calls the model's Outputs methods and derivatives methods to compute the model's outputs and derivatives at successive intervals within the major time step
 - Model outputs methods and derivatives methods -> invoke their corresponding system methods -> invoke the blocks outputs and derivatives (order specified by the outputs and derivatives methods execution lists generated in the link phase)

3. Optionally checks for discontinuities in the continuous states of the blocks

- Zero-crossing detection is used to detect discontinuities in continuous states

4. Computes the time for the next time step

- Steps 1 to 5 are repeated until the simulation stop time is reached



3.5 Solver

- Finds an approximate solution for a set of model equations

Fixed Step Size	Variable Step Size
• Δt is constant • if Δt is too large, simulation results can have a large error	• step solvers iterate to reach a solution based on an error tolerance • Δt changes from one simulation step to the next depending on the estimated error • Smaller time steps increase the accuracy of the simulation • variable step is chosen to minimize computational workload • ensures that all model states are computed to the accuracy specified by the user